CDLE · FORMATS

# Data Formats in Depth

*How each format lays out bytes, and what that costs a query*

A self-contained reading guide: read it before the format benchmarks, and come back to it when a result surprises you.

**Parquet**
Columnar layout, encodings, statistics and nested data

**Compression**
Codecs, levels and splittable files

**Avro**
Binary rows, schema evolution and the schema registry

**Text & friends**
CSV, JSON, ORC, Arrow, and how to choose

# Contents

*Six sections, one running example: the events table follows you through every format*


## 01 Foundations

Rows or columns
Splits and tasks
What bytes cost
The events table

## 02 Parquet

File anatomy and footer
Encodings
Statistics and pruning
Nested columns
Row-group and file size

## 03 Compression

snappy, zstd, gzip, lz4
Splittable or not

## 04 Avro

The container file
Binary encoding
Schema evolution
Schema registry in Kafka

## 05 Text & friends

CSV and JSON at scale
ORC and Arrow

## 06 Choosing

Six formats side by side
One pipeline, three formats
Predict before you measure
Takeaways and sources

# Three ideas this guide builds on

*Everything else in the guide follows from these; each one returns on almost every page*

## 1 · Rows or columns

A **row** layout stores each record's fields together: cheap to append, cheap to read whole records.

A **columnar** layout stores each column's values together: a query that needs 3 of 48 columns reads about 3/48 of the data.

## 2 · Files are read in splits

Spark cuts large files into **splits** of about 128 MB and hands each split to a task.

A format is **splittable** if a reader can start mid-file and find the next record boundary. If it cannot, one task reads the whole file.

## 3 · Bytes cost time and money

On object storage such as S3, every request and every byte read crosses the network and is billed.

A format earns its place by letting the reader **skip bytes**: columns it does not need, blocks that cannot match.




Three records, three columns. A query on **day** reads one contiguous run in the columnar layout; in the row layout it must read past every id and total to reach it.

**NEW TERMS**

**Spark** engine that runs a job as parallel tasks
**Task** one unit of work: one split, one core
**S3** Amazon's object store, read over HTTP
**Record** one row: all fields of one event

# One table, followed through every format

*The events table: clickstream records from a web shop, used in every example of this guide*

**events — 9 of its 48 columns**

```
event_id    STRING      # unique per event
ts          TIMESTAMP
day         DATE        # one value per day
country     STRING      # ~200 distinct values
user_id     BIGINT
device      STRING      # web | ios | android
total       DECIMAL(10,2)
items       ARRAY<STRUCT<sku: STRING, qty: INT>>
notes       STRING      # usually null
...         39 more columns
```

## Three ways it is used

### Q1 · Analytics

`SELECT day, SUM(total) ... WHERE country = 'PT' GROUP BY day` reads 3 of 48 columns and a small share of the rows.

### Q2 · Replay

Reads every column of one hour of events, to re-run them through a corrected pipeline.

### Q3 · Ingest

Appends new events continuously as they arrive from Kafka.

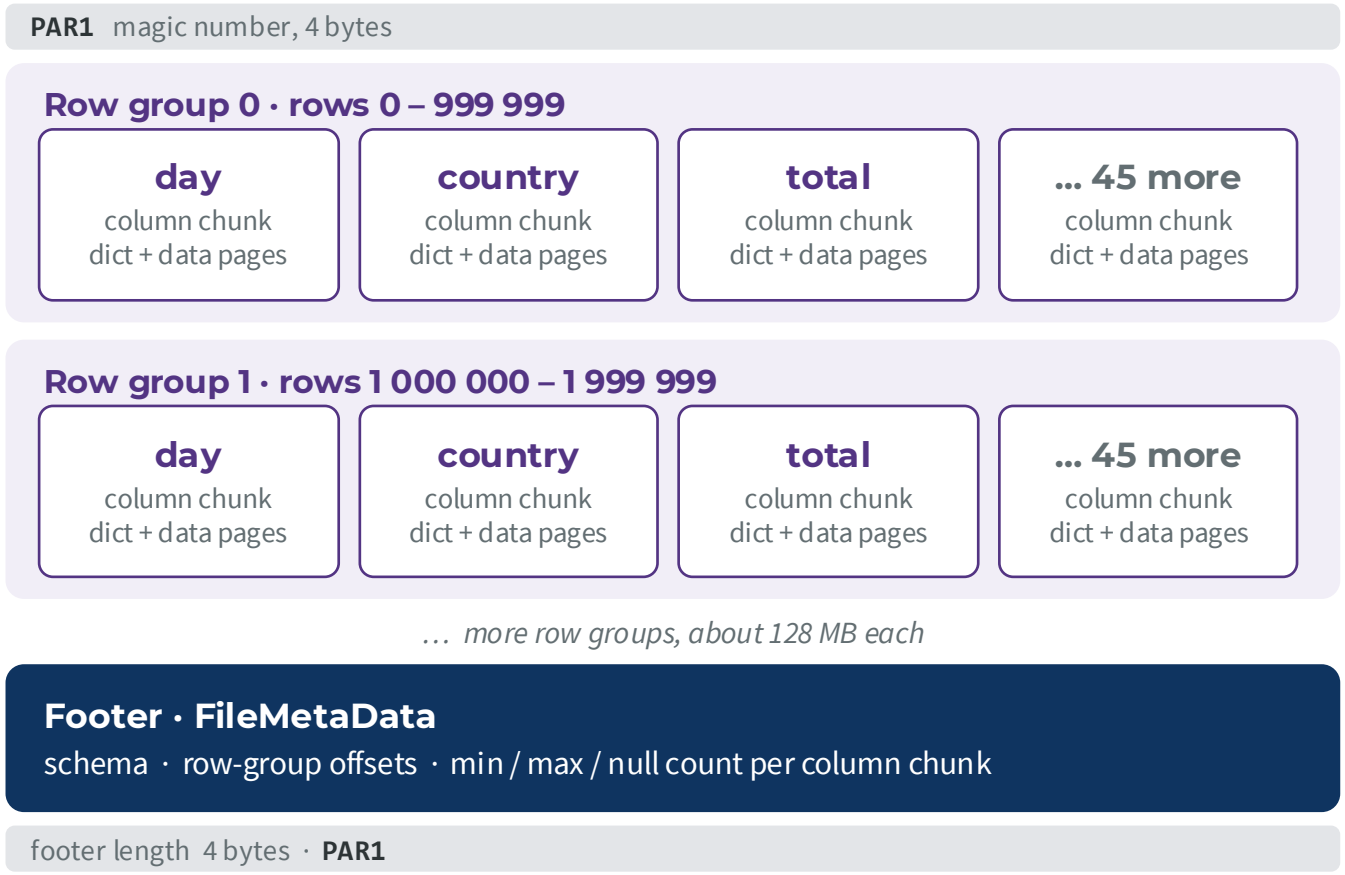
**Scale (illustrative):** 10 billion rows, about 2 TB as Parquet. Keep Q1–Q3 in mind: no format is the best choice for all three.

**NEW TERMS**

- **Clickstream** log of a site's clicks and page views
- **Kafka** distributed log that streams events
- **Cardinality** number of distinct values in a column
- **DECIMAL(10,2)** exact number: 10 digits, 2 decimals
- **ARRAY<STRUCT>** a list of nested records in one field
- **Parquet** columnar file format, pages 5–11

# Parquet — the map lives at the end

*A file is a set of row groups; each holds one column chunk per column, split into pages. The footer says where all of them are*




## How a reader opens a file

**1** · Read the last 8 bytes: the footer length and the magic word PAR1.

**2** · Read the footer: schema, where each row group and column chunk starts, and statistics per chunk.

**3** · Decide which row groups and which column chunks the query needs.

**4** · Fetch only those byte ranges. On S3, each one is a ranged GET request.

*For Q1 that means 3 of 48 column chunks per row group, and only the row groups whose statistics allow country = 'PT'.*

**NEW TERMS**

- **Row group** horizontal slice of rows, about 128 MB
- **Column chunk** one column's values in one row group
- **Page** unit of encoding and compression, ~1 MB
- **Footer** metadata block at the end of the file
- **Magic number** fixed bytes that mark the file type
- **Ranged GET** S3 request for one byte range of a file

Source: Apache Parquet format specification, “File Format” (parquet.apache.org/docs/file-format). Row-group size shown is the Spark default.

# Parquet is written row group by row group, footer last

*The writer buffers rows per column in memory, flushes a row group at about 128 MB, and writes the footer only when the file is closed*

### 1 · Rows arrive

`id · day · country · total`

`id · day · country · total`

`id · day · country · total`

from Spark tasks, one at a time

### 2 · Buffer per column

day | ctry | total | …

values encoded into pages of about 1 MB as each buffer fills

### 3 · Flush a row group

**row group n**
all column chunks

written when the buffers reach about 128 MB; memory is then reused

### 4 · Close the file

**footer**
+ length + PAR1

schema, row-group offsets and statistics, known only at the end

### The file on disk, in the order it is written




1st flush | 2nd flush | 3rd flush | close()

### Memory per open file

Each task holds one row group of buffers for every file it has open. Writing many partitions at once multiplies that memory.

### No footer, no file

A writer that crashes before close() leaves a file without a footer, which no reader can open. Table formats commit a file only after it is closed.

### Files are never appended

A finished file cannot grow. New rows mean new files, so frequent small writes leave many small files to compact later.

**NEW TERMS**

- **Buffer** memory that collects data before writing
- **Flush** write buffered data out to the file
- **Table format** layer (Delta, Iceberg) tracking a table's files
- **Commit** make written files visible all at once
- **Compaction** rewrite many small files as fewer large ones

Sizes are Spark defaults (parquet.block.size, parquet.page.size). Source: Apache Parquet specification, “File Format”.

# Encodings shrink a column before compression

*Parquet re-encodes each column's values to exploit repetition, then compresses the result page by page*

### 1 · Raw values

`PT PT PT ES ES PT FR FR`

Each value stored in full: 8 strings, each with a 4-byte length prefix.

→

### 2 · Dictionary

`0=PT 1=ES 2=FR`

`0 0 0 1 1 0 2 2`

Distinct values go once into a dictionary page; the data page keeps small integer IDs.

→

### 3 · RLE / bit-packing

`0×3 1×2 0×1 2×2`

IDs are stored as runs, or packed at 2 bits each: 8 IDs fit in 2 bytes.

→

### 4 · Compression

`page → snappy / zstd`

The codec compresses each encoded page; encoding has already removed most of the repetition.

*The country column of 8 rows, step by step. The same three encodings below cover most real columns.*

## Dictionary

`RLE_DICTIONARY` · the default for most columns.

Works when a column has few distinct values: country, device, status.

Past its limit (1 MB in parquet-java, the Java library Spark uses to write Parquet; formerly parquet-mr), the writer falls back to plain encoding.

## Delta

`DELTA_BINARY_PACKED` · for integers and timestamps.

Stores the difference between consecutive values, then bit-packs the differences.

A sorted ts column shrinks to a few bits per value.

## Byte-stream split

`BYTE_STREAM_SPLIT` · for floats and doubles.

Puts byte 1 of every value together, then byte 2, and so on.

Saves nothing by itself, but lets the codec compress measurements such as sensor readings much better.

**NEW TERMS**

- **Encoding** compact form of values, before compression
- **Dictionary page** the distinct values of one column chunk
- **RLE** run-length encoding: value + repeat count
- **Bit-packing** store small integers in fewer bits
- **Codec** compression algorithm, e.g. snappy, zstd
- **Plain encoding** every value stored in full

Source: Apache Parquet format specification, “Encodings”. Limits are parquet-java defaults.

# Statistics let a reader skip what cannot match

*Every column chunk records its min and max. Q1 can reject row groups after reading only the footer, if the data is sorted*

**Unsorted by country** every range contains 'PT': 8 of 8 row groups read

| RG 0 | RG 1 | RG 2 | RG 3 | RG 4 | RG 5 | RG 6 | RG 7 |
|---|---|---|---|---|---|---|---|
| **AR – VN** | **BE – US** | **AT – ZA** | **CA – PT** | **BR – SE** | **AU – UY** | **DE – TR** | **AL – ZW** |

**Sorted by country** only 2 ranges can contain 'PT': 2 of 8 row groups read

| RG 0 · skipped | RG 1 · skipped | RG 2 · skipped | RG 3 · skipped | RG 4 · skipped | RG 5 | RG 6 | RG 7 · skipped |
|---|---|---|---|---|---|---|---|
| **AD – BR** | **BR – DE** | **DE – FR** | **FR – IT** | **IT – NL** | **NL – PT** | **PT – SE** | **SE – ZW** |

## What makes pruning work

- Pruning works per row group, and per page when the writer adds the optional **page index**.
- It applies to =, <, >, `BETWEEN`, `IN` and `IS NULL` filters on columns that have statistics.
- **Sort or cluster** on the columns you filter by before writing: `df.sortWithinPartitions("country")`.
- Readers can also check the **dictionary**: if 'PT' is not in it, the row group is skipped even when min/max allow it.

## Partitioning skips earlier

A path such as `day=2026-03-01/` lets Spark skip whole directories before any footer is read.

Partition on a low-cardinality column you always filter by (day); sort inside the files on the next one (country).

**NEW TERMS**

- **Pruning** skipping data a filter proves irrelevant
- **min / max** smallest and largest value per chunk
- **Page index** optional min / max for every page
- **Partitioning** one directory per value of a column
- **Clustering** storing similar values close together
- **Low cardinality** few distinct values, e.g. day

Ranges are illustrative. Source: Apache Parquet format specification, “Metadata” and “Page Index”.

# Two skips cut Q1 from 2 TB to about 31 GB

*Q1 sums total per day for country = 'PT'. Column projection and row-group pruning decide which bytes leave S3*

### Bytes read from S3 for Q1, drawn to scale

**0 · Read the footers**
schema and statistics, before any data

**a few MB** the only bytes read before deciding

**1 · Stored on S3**
48 columns, about 16 000 row groups




**2 · Column projection**
3 of 48 columns: day, country, total



**≈ 125 GB · 1/16 of the table**

**3 · Row-group pruning**
sorted by country: 2 of 8 row groups



**≈ 31 GB · 1/64 of the table**

## What still happens to the 31 GB

- Pages are decompressed and decoded back into values.
- Statistics only prove a row group **may** hold 'PT', so each row is still filtered.
- Survivors are summed per day.

## What breaks the skips

- `SELECT *` reads all 48 columns again.
- Unsorted data: every row group's range contains 'PT', so 8 of 8 are read.
- A filter on an expression, such as `upper(country) = 'PT'`, cannot use the statistics.

**NEW TERMS**

**Projection** reading only the columns a query names
**Predicate** a filter condition, e.g. country = 'PT'
**Pushdown** giving the filter to the reader to skip data
**Decode** turn encoded bytes back into values

Illustrative arithmetic: assumes equal-sized columns and the sorted layout from the statistics page. Real columns differ in size.

# Nested columns flatten into two small numbers

*Parquet stores items.sku as one plain column; repetition and definition levels record where each list starts and what is missing*

## Three records and their schema

```
r1 {items: [{sku: "A"}, {sku: "B"}]}
r2 {items: []}
r3 {items: [{sku: "C"}]}

# schema (simplified)
repeated group items {
  required binary sku
}
```

## Column items.sku as stored

| value | r | d |
|---|---|---|
| **A** | 0 | 1 |
| **B** | 1 | 1 |
| **(none)** | 0 | 0 |
| **C** | 0 | 1 |

## Reading the levels

**r** · repetition level: 0 starts a new record; 1 adds another element to the current list.

**d** · definition level: how many optional or repeated levels are present. d = 1 means a value exists; d = 0 means the list is empty.

## Why it matters

- A query on `items.sku` reads only that column, not the whole nested structure.
- The levels are tiny and RLE-encode well, because most records repeat the same pattern.
- Spark and DuckDB rebuild the nesting from r and d. Row formats such as Avro and JSON store each record's lists inline, so reading one nested field means decoding every record.

**NEW TERMS**

**Schema** the field names and types of the data
**Struct** a group of named fields in one value
**Repeated field** a field that holds a list
**Optional field** a field that may be missing (null)
**Dremel** Google's 2010 system; Parquet copies its nesting
**DuckDB** in-process analytical SQL engine

Source: Melnik et al., “Dremel: Interactive Analysis of Web-Scale Datasets”, VLDB 2010; Apache Parquet format specification, “Nested Encoding”.

# Size decides parallelism and skipping

*Larger units compress and scan better; smaller units skip more finely. The defaults are a good starting point*

| Unit | Default in Spark | Too small | Too large |
|---|---|---|---|
| **Row group** | 128 MB · `parquet.block.size` | more footers and seeks; weaker compression | fewer tasks; coarser skipping; more writer memory |
| **Page** | 1 MB · `parquet.page.size` | more page headers to read | coarser page-level skipping |
| **File** | one or more per write task | **small-files problem**: thousands of footers and S3 requests | too few files to read in parallel |

## Writing the events table well

```
(events
  .repartition("day")               # one task per day
  .sortWithinPartitions("country") # tight min/max
  .write
  .partitionBy("day")               # day=2026-03-01/
  .option("parquet.block.size", 128 * 1024 * 1024)
  .parquet("s3a://lake/events/"))
```

## Rules of thumb

- Aim for files of roughly **128 MB to 1 GB**: a 2 TB table becomes 2 000 to 16 000 files.
- Streaming jobs write many small files: **compact** them on a schedule (Delta `OPTIMIZE`, Iceberg `rewrite_data_files`).
- Check what you wrote: DuckDB's `parquet_metadata('f.parquet')` or PyArrow's `ParquetFile(f).metadata` lists row groups and statistics.

**NEW TERMS**

**Small-files problem** overhead of too many tiny files
**Delta Lake** table format on Parquet with ACID commits
**Iceberg** open table format for very large tables
**OPTIMIZE** Delta command that compacts files
**PyArrow** Python library for Arrow and Parquet
**repartition** redistribute rows across tasks

# Codecs trade CPU for bytes

*snappy decompresses fastest; zstd usually gives clearly smaller files and still reads quickly; gzip is small but slow*

| Codec | Speed | Files | Where you meet it | Watch out for |
|---|---|---|---|---|
| **snappy** | very fast | larger | Spark's default for Parquet and Avro | no levels to tune |
| **zstd** | fast · levels 1–22 | smaller | Parquet, Avro, ORC, Kafka | high levels slow writes, not reads |
| **gzip** | slow | small | legacy Hive tables, `.csv.gz` exports | a gzipped text file is not splittable |
| **lz4** | very fast | larger | Kafka, ORC, Parquet `LZ4_RAW` | Parquet's older `LZ4` codec is deprecated |

### Choosing the codec in Spark

```
spark.conf.set("spark.sql.parquet.compression.codec",
               "zstd")
df.write.option("compression", "zstd").parquet(path)

# Avro calls the same codec "zstandard"
df.write.format("avro") \
  .option("compression", "zstandard").save(path)
```

### Where compression happens

- Parquet compresses **each page** after encoding; Avro compresses **each block**.
- The file structure stays readable around the compressed parts, so both formats stay splittable.
- Measure on your own data: ratios depend on how repetitive the values are.

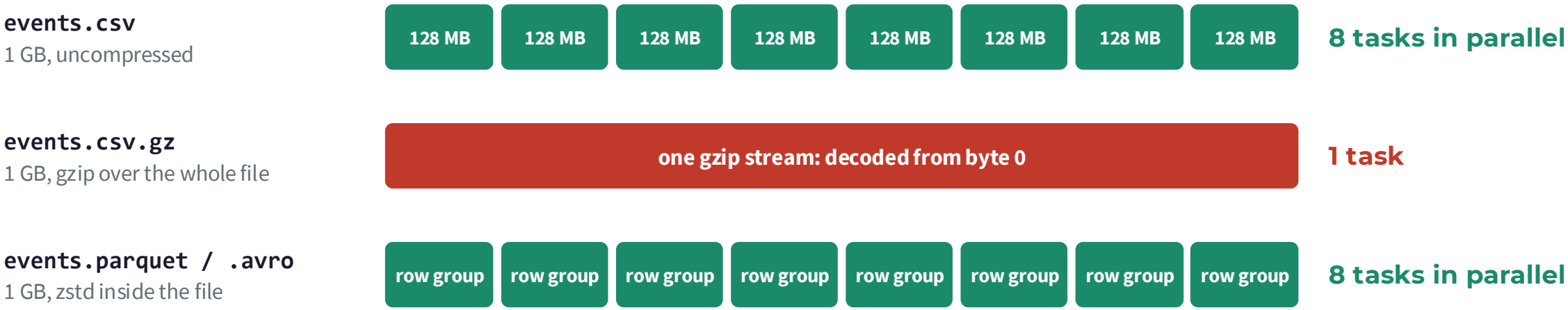
**NEW TERMS**

**snappy** Google codec: very fast, larger files
**zstd** Zstandard: tunable speed against size
**gzip** older codec: small files, slow
**lz4** very fast codec, common in Kafka
**Level** knob trading CPU time for smaller files
**Hive** SQL data warehouse built on Hadoop

Speed and size are relative, for typical tabular data. Sources: Apache Parquet specification, “Compression”; Spark SQL guide, Parquet and Avro data sources.

# Splittable files are read by many tasks at once

*A reader can start mid-file only if it can find a record boundary there; where compression is applied decides that*




## Text files

CSV and JSON Lines split at newlines when uncompressed. A quoted field that contains a newline breaks this; Spark's `multiLine` option then reads each file whole.

## Whole-file codecs

gzip or zstd over a whole text file is not splittable; bzip2 is, but it is slow. One 10 GB `.csv.gz` becomes one 10 GB task.

## The fix

Decompress landing files once, or better, convert them to Parquet early. Parquet and Avro compress inside the file, so they always split.

**NEW TERMS**

- **Split boundary** a point where a reader can start
- **Compressed stream** must be decoded from its first byte
- **bzip2** slow codec whose files can be split
- **Landing file** raw file exactly as received
- **Delimiter** character that separates fields, e.g. a comma
- **multiLine** Spark option: records span several lines

Split size: spark.sql.files.maxPartitionBytes, 128 MB by default. Source: Spark SQL guide, “Generic File Source Options” and CSV/JSON data sources.

# Avro — schema in the header, data in blocks

*An Avro file is self-describing: the writer's schema travels with the data, and sync markers let readers split it*




*header · written once, when the file is created*

*data blocks · appended as records arrive*

**The schema stored in avro.schema (JSON)**

```
{"type": "record", "name": "Event",
 "fields": [
  {"name": "event_id", "type": "string"},
  {"name": "ts",
   "type": {"type": "long",
            "logicalType": "timestamp-millis"}},
  {"name": "country", "type": "string"},
  {"name": "total", "type": "double"},
  {"name": "notes", "type": ["null", "string"],
   "default": null}
 ]}
```

## Inside each block

A block holds a **count** of records, their **size** in bytes, the records themselves (compressed with the header's codec), then the file's sync marker.

- **Row-oriented:** whole records sit together, so appends (Q3) and full-record reads (Q2) are cheap.
- **Splittable:** a task jumps to any byte offset and scans forward to the next sync marker.
- **No statistics:** every query reads every block, so Q1 pays for all 48 columns.

**NEW TERMS**

**Avro** row-based binary format with a schema
**Self-describing** the file carries its own schema
**Block** group of records written together
**Sync marker** 16 random bytes that end every block
**logicalType** annotation, e.g. a long that is a timestamp
**Union** field holding one of several types

Source: Apache Avro specification, “Object Container Files” (avro.apache.org/docs).

# Avro records carry no field names

*Values are written in schema order as compact binary; without the writer's schema, the bytes cannot be decoded*

**One record · fields event_id, qty, country, notes**

```
{"event_id": "e7", "qty": 3,
 "country": "PT", "notes": null}

# as JSON text: 53 bytes
```

**The same record as Avro binary: 8 bytes**

| 04 | 65 | 37 | 06 | 04 | 50 | 54 | 00 |
|---|---|---|---|---|---|---|---|
| event_id · length 2, then 'e' '7' | | | qty · 3 → 6 | country · length 2, 'P' 'T' | | | notes · null |

## The encoding rules

- **int, long:** zig-zag, then variable length: 3 becomes 6, stored in one byte.
- **string, bytes:** a length, then the raw bytes.
- **union:** the index of the branch, then the value. `["null", "string"]` holding null is the single byte 00.
- **record:** fields in schema order and nothing else: no names, no separators.

## What follows from it

- **Compact:** 8 bytes here against 53 as JSON, before any compression.
- **Schema required:** a reader needs the writer's schema, from the file header or, in Kafka, from a schema registry.
- **Renaming is not free:** names exist only in the schema, so a rename needs an alias (next page).

**NEW TERMS**

**Zig-zag** maps negative ints to small positive ones
**Variable length** small integers take fewer bytes
**Schema order** fields in the order the schema lists them
**Alias** another accepted name for a field
**Schema registry** service that stores schema versions

Source: Apache Avro specification, “Binary Encoding”.

# Readers and writers may use different schemas

*Avro matches the writer's schema against the reader's at read time, field by field, by name*

**Writer schema · v1, in old files**

```
{"name": "Event", "fields": [
 {"name": "event_id", "type": "string"},
 {"name": "country",  "type": "string"},
 {"name": "qty",      "type": "int"},
 {"name": "coupon",   "type": "string"}
]}
```

**resolved at read time**

**qty**
int → long: promoted

**coupon**
not in v2: ignored

**device**
not in v1: default 'web'

**Reader schema · v2, in new code**

```
{"name": "Event", "fields": [
 {"name": "event_id", "type": "string"},
 {"name": "country",  "type": "string"},
 {"name": "qty",      "type": "long"},
 {"name": "device",   "type": "string",
  "default": "web"}
]}
```

| Change to the schema | What the reader does with old data | Safe? |
|---|---|---|
| **Add a field with a default** | fills in the default | yes |
| **Add a field without a default** | fails: there is no value to use | no |
| **Remove a field** | ignores the writer's value | if no reader needs it |
| **Widen a type: int → long, float → double** | promotes the value | yes |
| **Rename a field** | matches only through `aliases` in the reader schema | with an alias |
| **Change a type: string → int** | fails | no |

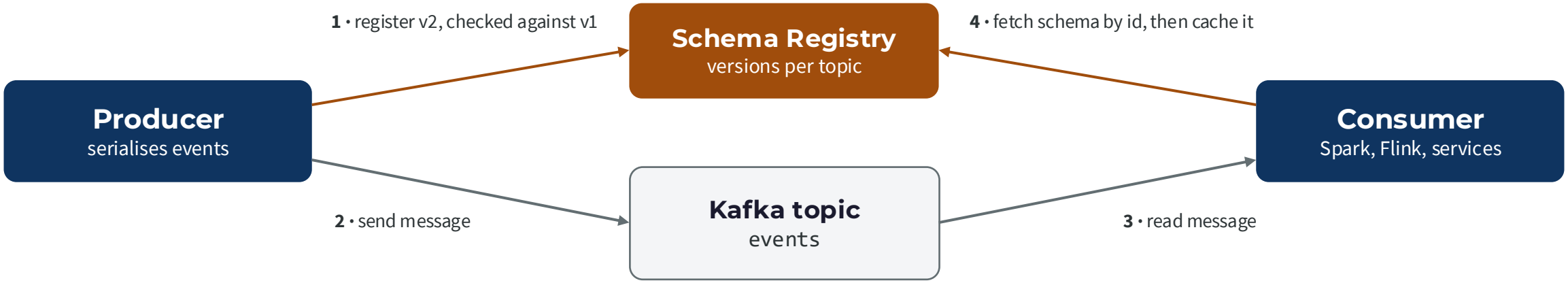
**NEW TERMS**

**Writer schema** schema the data was written with
**Reader schema** schema the reading code expects
**Resolution** matching the two schemas at read time
**Promotion** widening a type, e.g. int → long
**Default** value used when a field is missing

Source: Apache Avro specification, “Schema Resolution”.

# A schema registry enforces compatibility

*In Kafka, each message carries a 5-byte prefix that points to a registered schema, instead of the schema itself*




## One message on the wire

| 0x00 | schema id | Avro binary record |
|---|---|---|
| magic byte | 4 bytes | the 8-byte encoding from two pages back, with no schema attached |

### BACKWARD · default

The new schema reads data written with the previous one. Upgrade consumers first.

### FORWARD

The previous schema reads data written with the new one. Upgrade producers first.

### FULL

Both directions hold: add or remove only fields that have defaults.

### _TRANSITIVE variants

Each mode can be checked against every earlier version, not only the last.

**NEW TERMS**

- **Producer** program that writes messages to Kafka
- **Consumer** program that reads messages from Kafka
- **Topic** named stream of messages in Kafka
- **Magic byte** first byte; marks the message format
- **Schema id** registry number of one schema version
- **Compatibility** rule a new version must pass

Source: Confluent Schema Registry documentation, “Schema Evolution and Compatibility” and “Wire format”.

# CSV and JSON are easy to write, costly to read

*Text formats suit landing zones and data exchange; every query that reads them pays to parse them*

## CSV

- **No types:** everything is text until parsed; `inferSchema` costs an extra pass over the data.
- **Fragile:** delimiters, quotes and newlines inside fields need their options set exactly.
- **No statistics:** every query reads and parses every byte, even for one column.

## JSON

- **Names in every record:** the 53-byte record from the Avro page, repeated 10 billion times.
- **JSON Lines** (one object per line) splits across tasks; one large JSON document (`multiLine`) does not.
- **Bad records:** `mode` PERMISSIVE keeps them in `_corrupt_record`; DROPMALFORMED or FAILFAST drop or stop.

**Read text once, with a declared schema**

```
schema = ("event_id STRING, ts TIMESTAMP, "
          "country STRING, total DOUBLE")

raw = (spark.read
         .schema(schema)     # no inference pass
         .option("header", True)
         .option("mode", "FAILFAST")
         .csv("s3a://landing/events/"))

# convert once, query the copy
raw.write.parquet("s3a://lake/events/")
```

**Pattern:** land data as text, convert it to Parquet once, and run every query on the Parquet copy. Keep the raw files for replay.

**NEW TERMS**

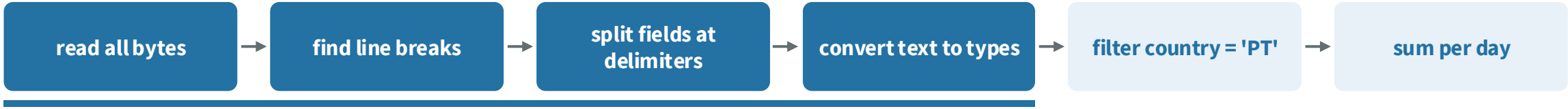
- **inferSchema** Spark guesses types with an extra scan
- **JSON Lines** one JSON object per line
- **Corrupt record** row the parser cannot read
- **FAILFAST** stop the job at the first bad record
- **Landing zone** storage for raw data as received
- **Replay** re-run old data through a new pipeline

Source: Spark SQL guide, CSV and JSON data sources (options header, inferSchema, mode, multiLine).

# Text formats make the CPU do more work per byte

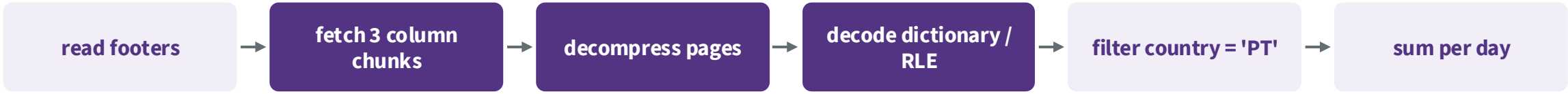
*The same Q1 on CSV and on Parquet: CSV parses every field of every row; Parquet decodes only what survived the skips*

## CSV · every query




**all 48 columns of every row**, as text

## Parquet · every query




**3 of 48 columns**, only row groups that may match

### Parsing runs on every value

Turning text such as "2026-03-01" or "12.40" into dates and decimals happens for every field read. Parquet stores values as typed binary.

### JSON parses the names too

Each record repeats its field names, and nested objects have to be walked, so the parser does more work per row than for CSV.

### See it in the Spark UI

Run the same query on both copies and compare Input size and task time on the stage page. The gap is the work above.

**NEW TERMS**

- **Parse** convert text into typed values
- **Typed binary** values stored in their native byte form
- **Vectorized reader** decodes a batch of values at once
- **Spark UI** web page of a job's stages and tasks
- **Stage** group of tasks that run between shuffles

Steps simplified. Spark's vectorized Parquet reader decodes values in batches of columns rather than row by row.

# ORC and Arrow complete the family

*ORC is Parquet's columnar sibling from the Hive world; Arrow is columnar in memory, for passing data between engines*

## ORC · columnar on disk

- A file is a series of **stripes**, ORC's equivalent of a Parquet row group: the unit a task reads or skips (64 MB by default in the ORC Java library, orc.stripe.size), then a file footer and a postscript.
- Each stripe holds **index data** with min/max every 10 000 rows, the column data and a stripe footer.
- Optional **bloom filters** answer "is this value here?" for high-cardinality columns.
- Required for **Hive ACID** tables; Spark and Trino read it natively.

## Arrow · columnar in memory

- A **memory layout** first: columns as contiguous buffers, grouped in record batches.
- Engines that share the layout pass data **without converting it**, zero-copy where layouts match.
- Spark uses Arrow for `toPandas()` and pandas UDFs when `spark.sql.execution.arrow.pyspark.enabled` is on.
- **Arrow IPC / Feather** files keep batches on disk for fast local reloads.




**NEW TERMS**

**Bloom filter** small structure: "maybe here" or "not here"
**Hive ACID** Hive tables that allow updates and deletes
**Trino** distributed SQL query engine
**Record batch** a chunk of rows held as column buffers
**Zero-copy** sharing memory without converting it
**pandas UDF** Python function run on Arrow batches

Sources: Apache ORC specification v1; Apache Arrow "Columnar Format"; Spark guide "Apache Arrow in PySpark".

# ORC — the same idea, organised in stripes

*An ORC file is a series of stripes, then a file footer and a postscript. Like Parquet, it is read from the end*

**ORC** magic, 3 bytes

**Stripe 0 · about 64 MB**

- **Index data** min / max per 10 000 rows (row index) · optional bloom filters
- **Row data** one or more streams per column: present, data, length, dictionary
- **Stripe footer** where each stream starts and how each column is encoded

**Stripe 1** index · row data · stripe footer

*… more stripes*

**File footer**
schema · stripe list · column statistics per stripe and for the file

**Postscript** codec · footer length · version

postscript length · 1 byte

## The same parts in Parquet

| ORC | Parquet |
|---|---|
| **stripe** | row group |
| **row index, every 10 000 rows** | page index |
| **stripe footer** | column-chunk metadata in the footer |
| **file footer** | footer (FileMetaData) |
| **postscript + 1 byte** | footer length + PAR1 |

**A reader starts at the last byte:** postscript length, then the postscript, then the file footer, and only then the stripes and streams it needs.

**NEW TERMS**

- **Stream** bytes for one part of one column
- **Present stream** marks which values are null
- **Length stream** sizes of variable-length values
- **Row index** min / max for every 10 000 rows

Source: Apache ORC specification v1 (orc.apache.org/specification/ORCv1). Stripe size is the ORC Java default.

# Six formats side by side

*Choose by how the data will be read; the columns that decide most cases are “Skips data?” and “Splittable”*

| Format | Layout | Splittable | Schema | Skips data? | Best for |
|---|---|---|---|---|---|
| **Parquet** | columnar · binary | yes, by row group | in the footer | yes: min/max, dictionary, page index | analytics at rest (Q1) |
| **ORC** | columnar · binary | yes, by stripe | in the footer | yes: indexes, bloom filters | Hive warehouses, ACID tables |
| **Avro** | row · binary | yes, by sync marker | in the header, with evolution rules | no | pipelines, Kafka, replay (Q2, Q3) |
| **CSV** | row · text | only if uncompressed | none | no | exports, data exchange |
| **JSON Lines** | row · text | only if uncompressed | field names in every record | no | landing zones, APIs |
| **Arrow IPC** | columnar · in memory | by record batch | in the stream header | no | passing data between engines |

**Parquet is the default for analytics at rest.** Every other row is an exception you choose on purpose, for a stage that writes or reads whole records.

# Four questions pick the format

*Ask them in order; the first "yes" decides. Most data at rest ends at the first question*

**1 · Will analytical queries read a few columns of many rows?**

yes → **Parquet**
ORC if your warehouse runs Hive ACID tables

no ↓

**2 · Is it a stream of whole records between services, with a schema that will change?**

yes → **Avro**
in Kafka, with a schema registry

no ↓

**3 · Must people or outside systems read it exactly as written?**

yes → **CSV or JSON Lines**
at the edge only; convert to Parquet on landing

no ↓

**4 · Is it passed in memory between engines or to Python?**

yes → **Arrow**
record batches; IPC / Feather files on local disk

**Four "no" answers?** Store it as Parquet: it is the default for data at rest, and every engine in this course reads it.

# A typical pipeline uses three formats

*Each stage picks the layout that matches how that stage reads and writes*




## Where the three queries of the events table run

### Q3 · Ingest → Avro on Kafka

Producers append compact rows; the registry rejects incompatible schema changes before they reach any consumer.

### Q2 · Replay → raw files or Avro

Full records in arrival order: re-run them through a corrected job without guessing what the old schema was.

### Q1 · Analytics → Parquet

Column pruning, statistics and compression, with the table partitioned by day and sorted by country.

**NEW TERMS**

**Lake** object storage that holds the tables
**Notebook** interactive code document, e.g. Jupyter
**Schema-checked** validated against the registry on write

Delta Lake stores its data as Parquet files; Apache Iceberg supports Parquet, ORC and Avro data files, with Parquet as the default.

# Predict before you measure

*Write an answer to each question before you benchmark the events table; then explain every gap between prediction and result*

## 1 · Size on disk

Rank CSV, JSON, Avro, Parquet + snappy and Parquet + zstd by bytes. Which two are closest, and why?

## 2 · One-column scan

For `SUM(total)` over all rows, how much less does Parquet read than CSV? Does the codec change the time?

## 3 · Selective query

For Q1, what share of row groups does Parquet read when the data is unsorted, and when it is sorted by country?

## 4 · Full-record read

For Q2, which reads every column, does Avro beat Parquet? What work does Parquet do that Avro does not?

## 5 · Write cost

Which format is slowest to write? Is the time spent encoding, compressing or sorting?

## 6 · Small files

Write the same data as 10 000 small Parquet files. What happens to Q1, and where does the time go?

# Key takeaways

*“A format decides which bytes a query must read. Choose it for how the data will be read, not for how it was produced.”*

### ✓ Layout follows the read

Columns for analytics; rows for pipelines and full-record replay.

### ✓ Parquet reads less in three ways

Projection skips columns, statistics skip row groups, encoding and compression shrink what is left.

### ✓ Sorting makes pruning work

Sort or cluster on the column you filter by; partition on the one you always filter by.

### ✓ Avro is about the schema

Compact rows, resolution rules for evolution, and a registry that enforces them in Kafka.

### ✓ Text formats are for landing

Declare the schema, convert to Parquet once, keep the raw files for replay.

### ✓ Codec and splitting are choices

snappy for speed, zstd for size; never query a large gzipped CSV directly.

# Sources

*Specifications first: every claim in this guide can be checked against one of these*

## Specifications

**Apache Parquet format specification**
File Format, Encodings, Nested Encoding, Page Index · parquet.apache.org/docs

**Apache Avro specification**
Object Container Files, Binary Encoding, Schema Resolution · avro.apache.org/docs

**Apache ORC specification v1**
orc.apache.org/specification

**Apache Arrow Columnar Format**
arrow.apache.org/docs/format/Columnar.html

**Zstandard and Snappy**
facebook.github.io/zstd · github.com/google/snappy

## Guides and papers

**Spark SQL guide, data sources**
Parquet, Avro, CSV, JSON, generic file options · spark.apache.org/docs/latest

**Spark, Apache Arrow in PySpark**
spark.apache.org/docs/latest/api/python

**Confluent Schema Registry documentation**
Schema Evolution and Compatibility; Wire format · docs.confluent.io

**Melnik et al., “Dremel”, VLDB 2010**
the origin of repetition and definition levels

**Kleppmann, Designing Data-Intensive Applications**
O'Reilly, 2017 · ch. 3 (storage) and ch. 4 (encoding, evolution)